

UNIT-2

Quick Work

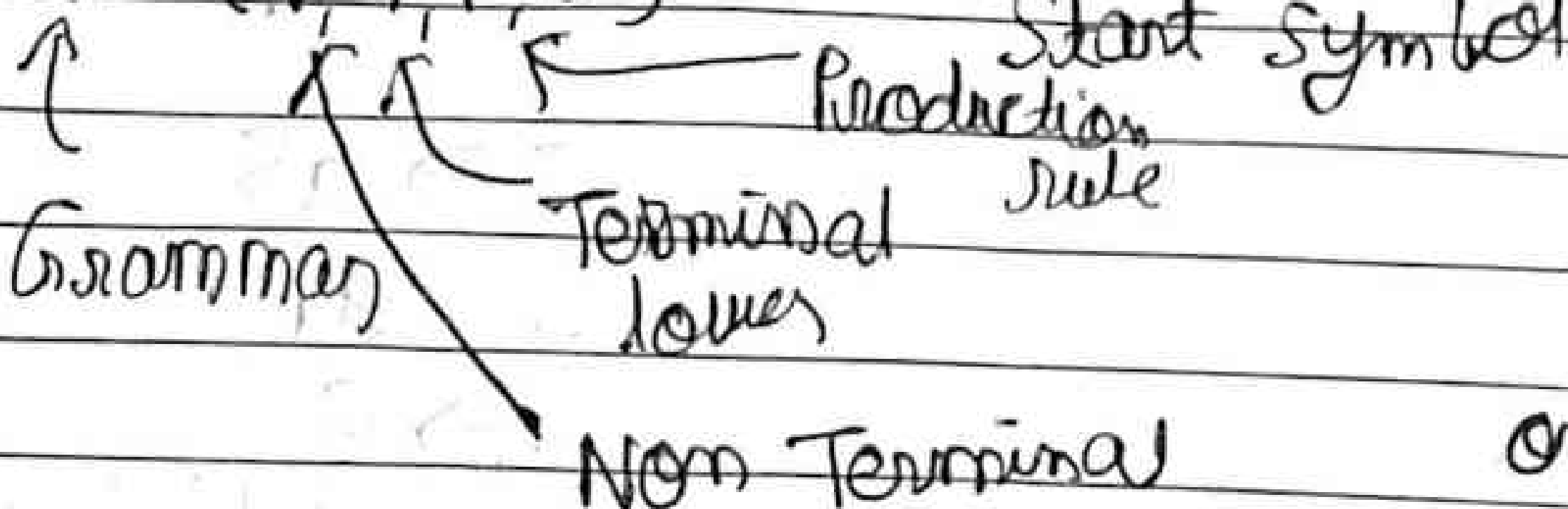
Page No.:

Date:

M T W T F S S

Context free Grammar -

$$G = (V, T, P, S)$$



$$S \rightarrow OS1$$

$$S \rightarrow \epsilon$$

Grammar

Terminal lower

Non Terminal

Production rule

Start symbol

$$\alpha \rightarrow \beta$$

only one variable

$$(VUT)^*$$

Context free

1) Type-2 grammar

2) Recognized by PDA

3) Production rules

$$A \rightarrow B$$

$A \in N$ (Non terminal)

$B \in V$ (Any string)

4)

4) Restriction is that less than regular grammar

5) Right hand of production has no restrictions

6) Intersection of 2 CFL need not to be a CFL

Regular Grammar

1) Type-3 grammar

2) Recognized by Finite Automata

3) Production are of the form

$$V \rightarrow VT/T \text{ (Left linear grammar)}$$

$$V \rightarrow TV/T$$

(Right linear grammar)

4) More than any other grammar

5) Right hand side of production should be either left linear or right linear

6) Intersection of 2 RG is RG

Closure Properties of Context Free language:

A set is closed under an operation if a set satisfies via these 3 operations:

- (i) Union
- (ii) Concatenation
- (iii) Kleene Star

(i) Union:

$$L_1 = \{a^n b^n c^m \mid m \geq 0 \text{ and } n \geq 0\}$$

$$L_2 = \{a^n b^m c^m \mid n \geq 0 \text{ and } m \geq 0\}$$

$$L_3 = L_1 \cup L_2 = \{a^n b^n c^m \cup a^n b^m c^m \mid n \geq 0, m \geq 0\}$$

is also context free

(ii) Concatenation:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

$$L_2 = \{c^m d^m \mid m \geq 0\}$$

$$L_3 = L_1 \cdot L_2 = \{a^n b^n c^m d^m \mid m \geq 0 \text{ and } n \geq 0\}$$

is also context free

(iii) Kleene Closure:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

$$L_1^* = \{a^n b^n \mid n \geq 0\}^* \text{ is also context free}$$

* Intersection & complementation are not closed

(i) Intersection:

$$L_1 = \{a^n b^n a^m \mid m, n \geq 1\} \text{ is context free language}$$

$$L_2 = \{a^n b^m a^m \mid m, n \geq 1\}$$

$$L_1 \cap L_2 = \{a^n b^n a^n \mid n \geq 1\} \text{ which is not a CFL}$$

(ii) Complement:

Let L_1 & L_2 be CFL
 L_1^c and L_2^c be CFL too (assume)
 $(L_1^c \cup L_2^c)^c$ is also CFL (Closure Property)
 But, using De Morgan's law,
 $(L_1^c \cup L_2^c)^c = L_1 \cap L_2$
 $\rightarrow$ But intersection is not closed

So assumption is wrong: & CFL is NOT closed under complementation.

Derivation -

- Sequence of production rules
- Used to get the input string through these production rules
- We have to take 2 decisions:
 - (a) We have to decide the non-terminal which is to be replaced
 - (b) We have to decide the production rule by which the non-terminal will be replaced
- We have 2 options to decide which non-terminal to be placed with production rule

leftmost derivation

rightmost derivation

Leftmost derivation -

Input is scanned & replaced with the production rule from left to right.
 The read input string from left to right

eg-

Production Rule

$$E = E + E$$

$$E = E - E$$

$$E = a + b$$

Leftmost derivation

$$E = E + E$$

$$E = (E - E) + E$$

$$E = a - b + E$$

$$E = a - b + E$$

$$E = a - b + a$$

Right most derivation - Input is scanned & replaced with the production

- We read the input string from right to left

eg-

Production Rule

$$E = E + E$$

$$E = E - E$$

$$E = a + b$$

Rightmost derivation

$$E = E - E$$

$$E = E - (E + E)$$

$$E = E - E + a$$

$$E = E - b + a$$

$$E = a - b + a$$

Q. Derive a string "abb" for leftmost derivation & right most derivation using a CFG given by

$$S \rightarrow AB/C$$

$$A \rightarrow aB$$

$$B \rightarrow Sb$$

Ans

Leftmost Derivation

S

$$[aB]B \quad (A=aB)$$

$$a[Sb]B \quad (B=Sb)$$

$$a[a]bB \quad (S \rightarrow \epsilon)$$

$$ab[Sb] \quad (B \rightarrow Sb)$$

$$ab[a]b \quad (S \rightarrow \epsilon)$$

$$abb$$

Rightmost Derivation

S

$$[a[Sb]] \quad (B=Sb)$$

$$a[a]b \quad (S=\epsilon)$$

$$[aB]b \quad (A=aB)$$

$$a[Sb]b \quad (B=Sb)$$

$$a[a]bb \quad (S=\epsilon)$$

$$abb$$

Q. Derive the string aabbabba for leftmost derivation using CF given by

$$S \rightarrow aB/bA$$

$$A \rightarrow a/aS/bAA$$

$$B \rightarrow b/bS/aBB$$

Ans

Leftmost Derivation

S

$$aB$$

$$a[aB]B$$

$$aabbB$$

$$aabbS$$

$$aabbabB$$

$$aabbabBS$$

$$aabbabba$$

$$aabbabba$$

Rightmost Derivation

S

$$aB$$

$$aabbB$$

$$aabbS$$

$$aabbabB$$

$$aabbabba$$

$$aabbabba$$

$$aabbabba$$

$$aabbabba$$

Q. Derive the string, 00101 using CFG given by

$S \rightarrow A|B$
 $A \rightarrow 0A| \epsilon$
 $B \rightarrow 0B|1B| \epsilon$

- # Derivation Tree / Parse Tree
- Graphical representation for given production rules for a given CFG.
 - Follows precedence of operators.
 - Deepest subtree traversed first.
 - Operator in the parent node has less precedence than the ~~same~~ operators in the subtree.
- Properties:
- Root node is always a node indicating start symbol.
 - Derivation is read from left to right.
 - Leaf node = terminal node.
 - Internal nodes are always the non-terminal nodes.

eg. Production Rule: $E \rightarrow E+E$

$E = E+E$
 $E = E+E$
 $E = a|b|c$

Ans

Step 1.

E

Step 2.

E

$E+E$

$E+E$

Step 3. $E \rightarrow E+E$

Step 4.

Step 5.

$E \rightarrow E+E$
 $E \rightarrow E+E$
 $E \rightarrow E+E$

$E * E$

$E * E$

$E * E$

a

a

b

a

b

c

eg

Draw a derivation tree for the string 11bbaabb from CFG given by $S \rightarrow bsb|ab|b$

Ans

S

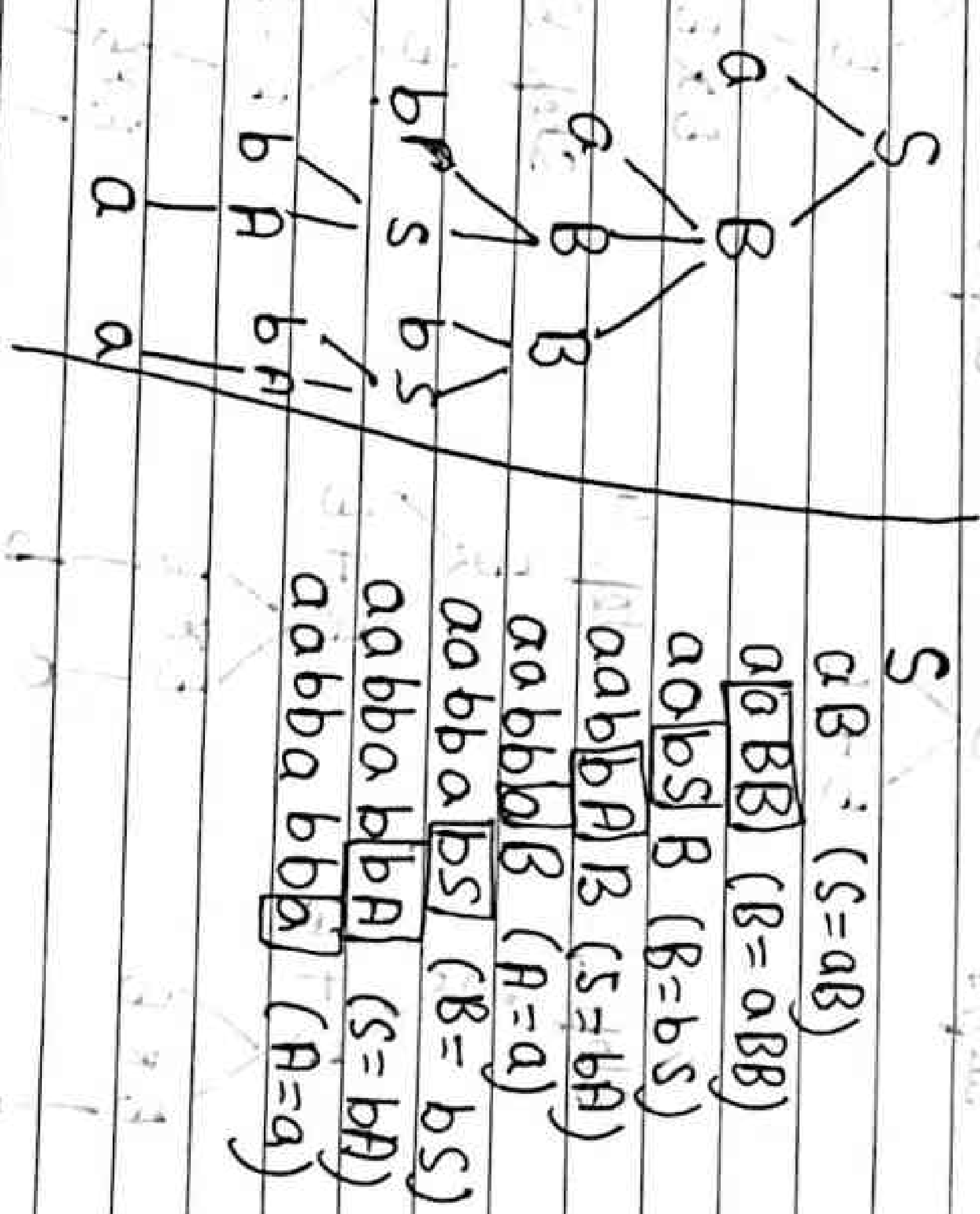
bsb

bsb

a

Q Construct a derivation tree for string $aabbabb$ a

Ans



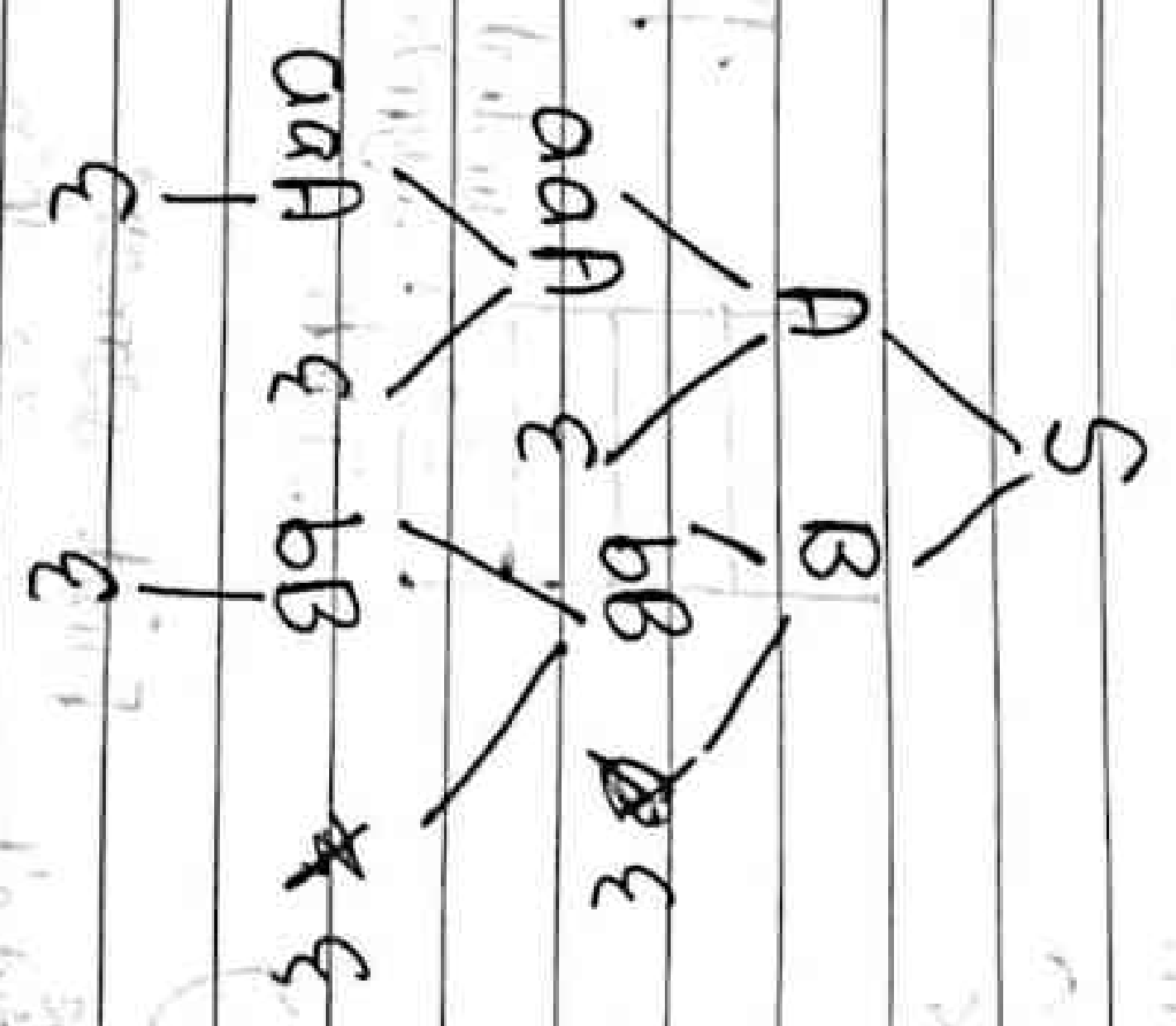
- # Purpose of parse tree -
- Visual representation of derivations
 - Ambiguity detection
 - Understanding structure of strings
 - Ambiguity resolution

Unambiguous Grammar -

- Does not contain more than one leftmost derivation / rightmost derivation / parse tree for the given string

Q $S \rightarrow AB$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$
 Check whether $w = aaaaabbb \in L(G)$

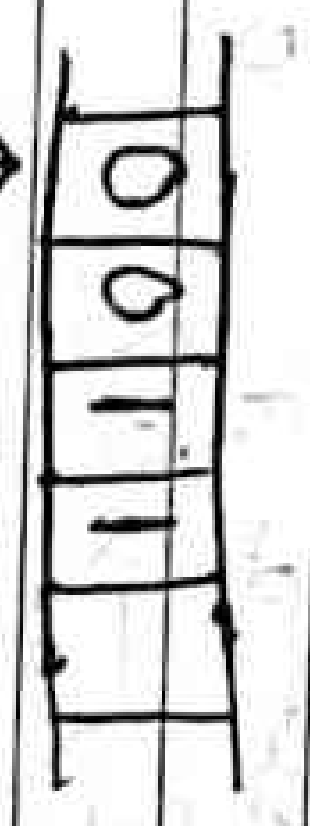
Ans



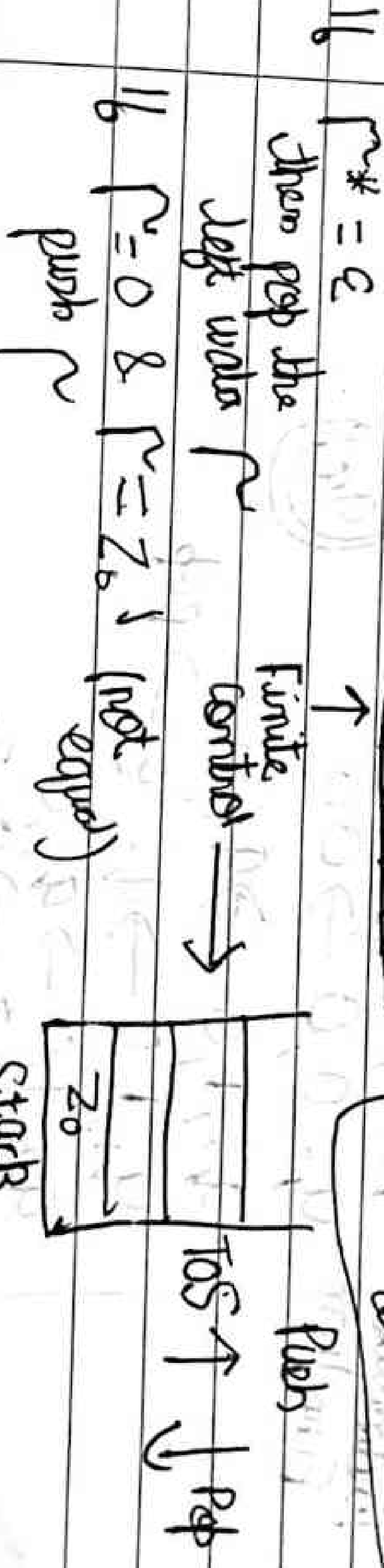
aaaaabbb

Pushdown Automata -

- $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, f)$
- Q = finite set of states like FA
- Σ = input symbol like FA
- Γ = stack alphabet (push into stack)
- δ = Transition function, $\delta: Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$
- q_0 = initial state
- Z_0 = stack start symbol
- F = final state



Input/Pop all 0 in stack & pop the 0s when 1 comes



$\Gamma = \text{law}$

PDA Designing -

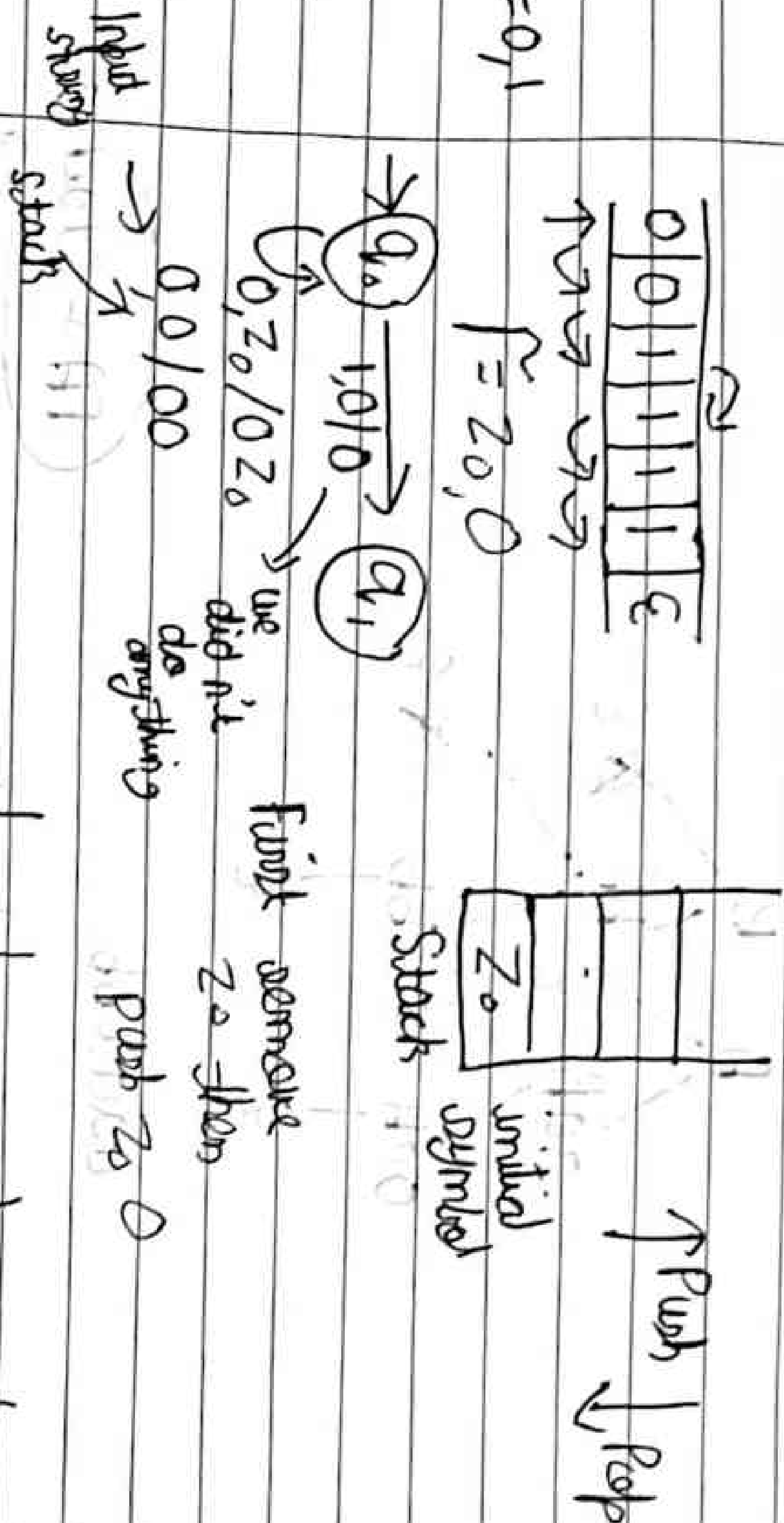
Q. $P = (\emptyset, \Gamma, \delta, q_0, z_0, F)$

$L = \{0^n 1^n, n > 0\}$

$011, 00111, 0001111, \dots$

$10, 1001, 100110$
 x, x, x, x

$\Sigma = 0, 1$
 $\Gamma = z_0, \emptyset$



Transitions
Function

q_0

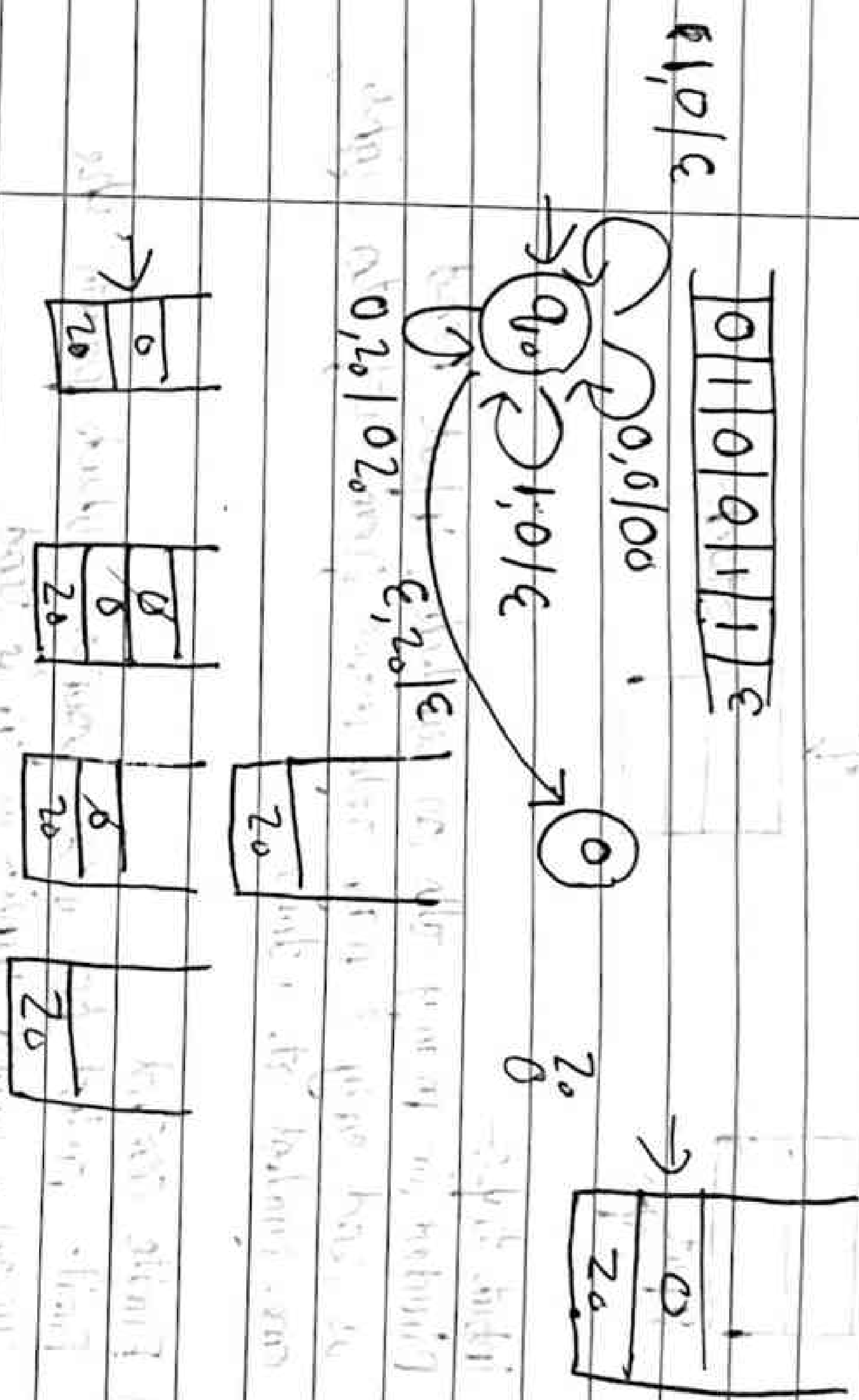
use pop one then 1

$q_0, 1, 0 \rightarrow q_1, 0$
 $q_1, 1, 0 \rightarrow q_1, \epsilon \rightarrow \text{pop}$
 $q_2, 1, 0 \rightarrow q_1, 0$
 $q_2, \epsilon, z_0 \rightarrow q_2, z_0$

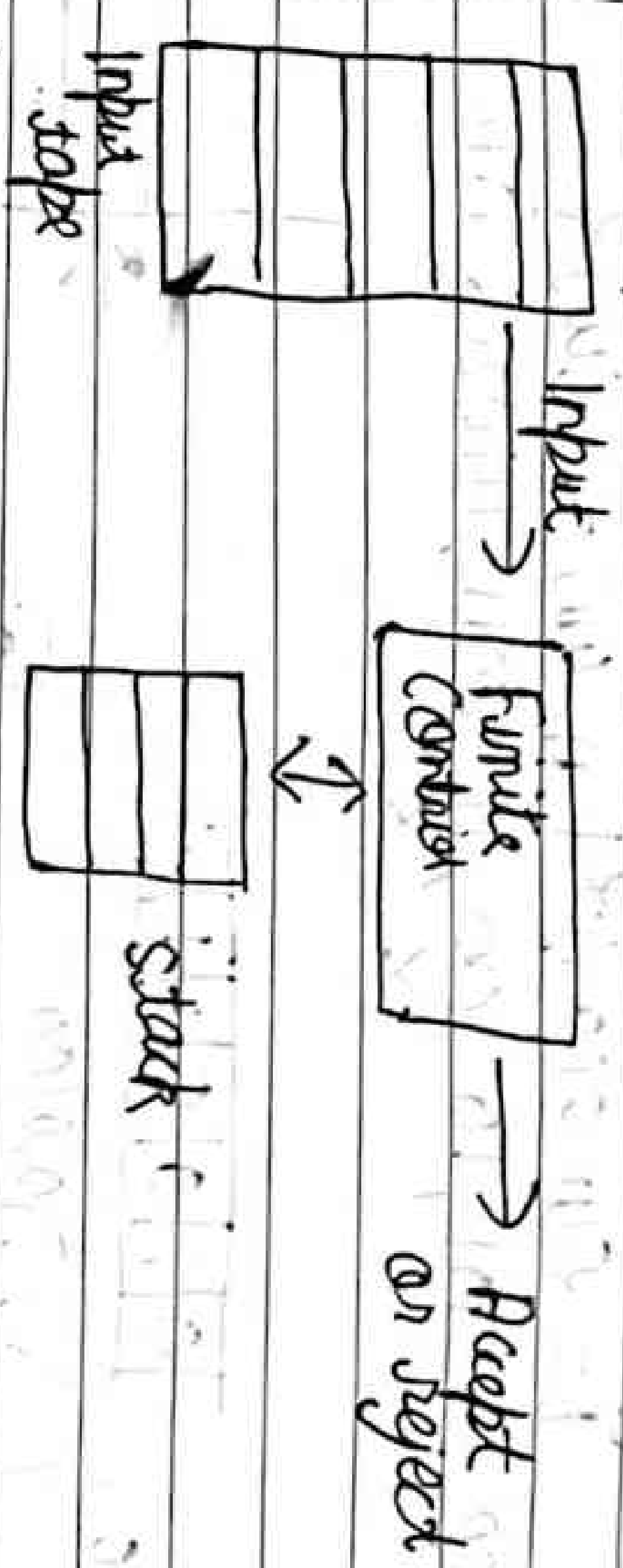
Q. $\Sigma = 0 \times (\epsilon \cup \epsilon) \times \Gamma \rightarrow 0 \times \Gamma^*$

$L = \{w \in \{0, 1\}^* \mid n_0(w) = n_1(w)\}$

Ans $01, 10, 00, 11, 0011, 11000, 0101$



PDA Components -



1) Input tape -

Divided in many cells or symbols. Input head is read only & may only move from left to right, one symbol at a time.

2) Finite control:

Finite control has some pointers which points the current symbol which is to be read.

3) Stack:

Structure in which we can push & remove the items from one end only. It has a infinite size. In PDA, stack is used to store elements temporarily.

* Transitions of PDA depends upon,

- (i) the current state
 - (ii) input symbol from input tape
 - (iii) symbol at top of stack
- PDA can also make a spontaneous transition using ϵ as its input instead of an input symbol.

Instantaneous Description (ID) -

- Instant notation of how a PDA computes a input string & make a decision that string is accepted or rejected.

- An instantaneous description is a triple (q, w, α) where

- q describes the current state
- w describes the remaining input
- α describes the stack contents.

Turnstile Notation -

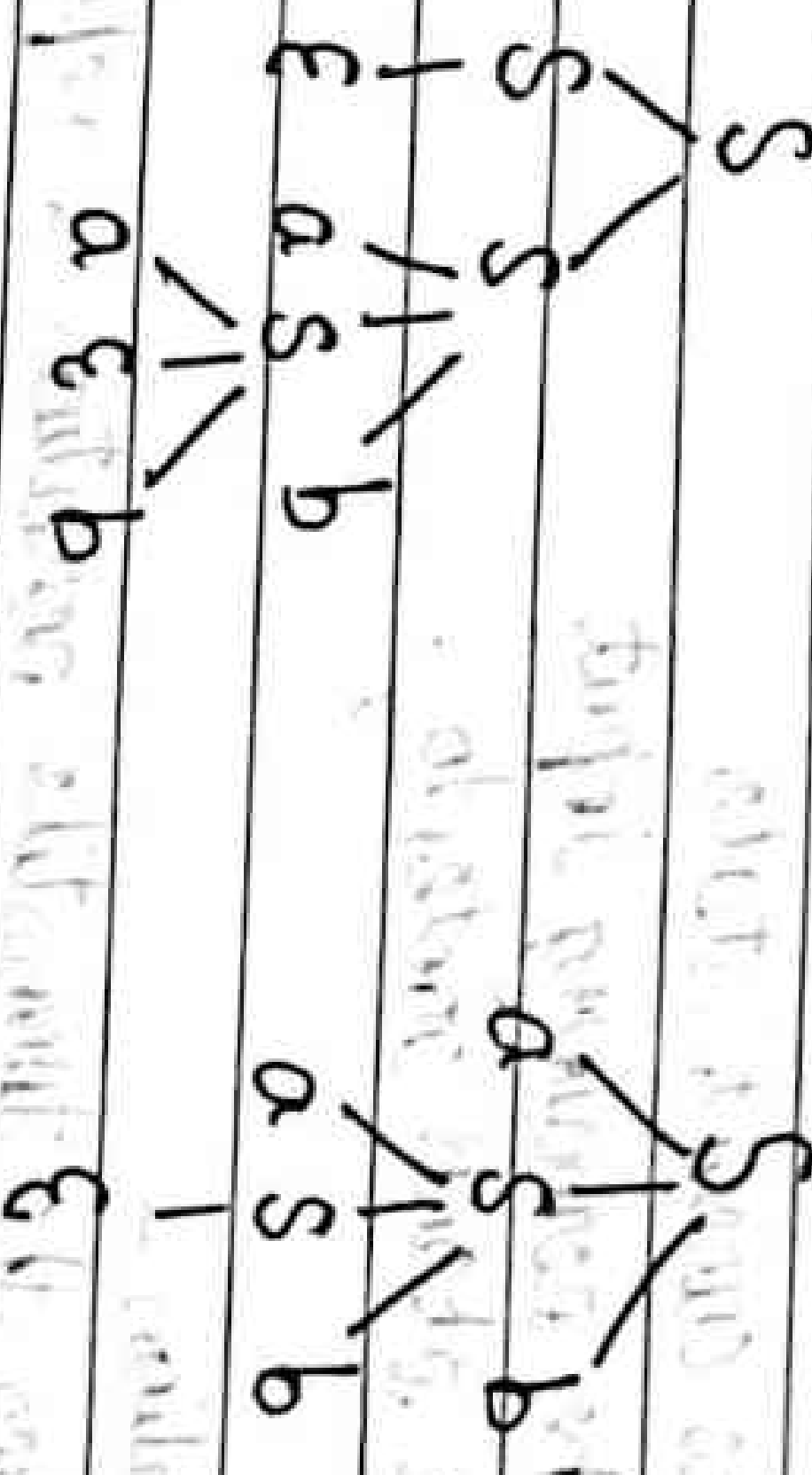
- sign describes the turnstile notation & represents move
- * sign describes a sequence of moves.

Q. Check whether the given grammar is ambiguous or not

$S \rightarrow aSb \mid SS \mid \epsilon$

"aabb" for this string

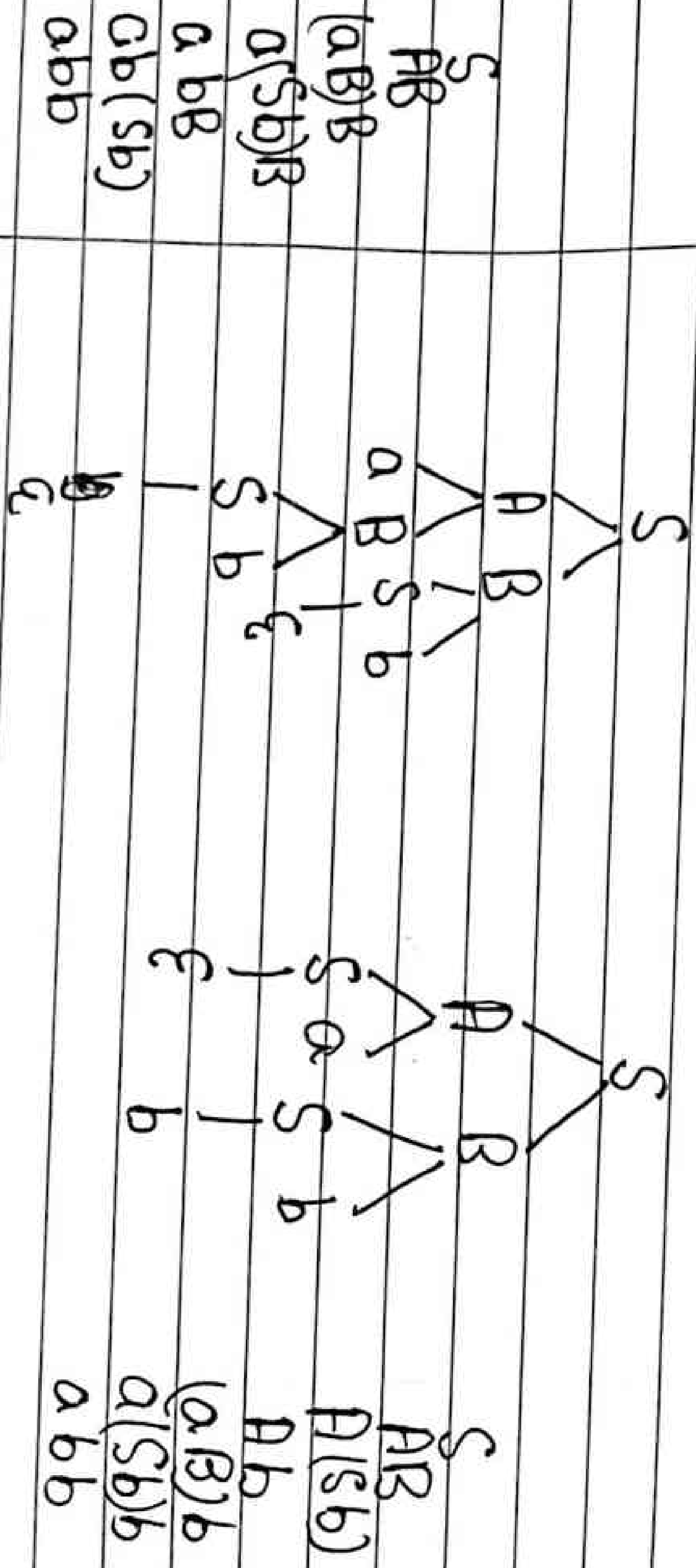
Ans



Ambiguous

Q. $S \rightarrow AB \mid \epsilon \mid b$
 $A \rightarrow aB \mid Sa$
 $B \rightarrow Sb$
"aabb"

Ans



Implications of Ambiguity -

- Causes syntactic uncertainty in compilers & interpreters.
- Leads to difficulty in defining semantics of a language
- Undecidable in programming languages

Inherently Ambiguous Languages -

- Some CFL languages are inherently ambiguous, meaning no unambiguous CFG can generate it.

- eg: $L = \{a^i b^j c^k \mid i=j \text{ or } j=k\}$

Chomsky's Normal Form (CNF)

- A CFG is in CNF if all production rules satisfy one of the following conditions:

- Start symbol generating ϵ . For example, $A \rightarrow \epsilon$.
- A non-terminal generating 2 non-terminals. For example, $S \rightarrow AB$.
- A non-terminal generating a terminal. For example, $S \rightarrow a$.

- eg:

- $G_1 = \{S \rightarrow AB, S \rightarrow c, A \rightarrow a, B \rightarrow b\}$
- $G_2 = \{S \rightarrow aA, A \rightarrow a, B \rightarrow c\}$

- Production rules of grammar G_1 satisfy the rules specified for CNF, as the grammar G_1 is in CNF.

- However, the production rule of grammar G_2 does not satisfy the rules specified for CNF as $S \rightarrow aA$ contains terminal followed by non-terminal. So the grammar G_2 is not in CNF.

$$\begin{aligned} Q. \quad & S \rightarrow a/aA/B \\ & A \rightarrow aBB/\epsilon \\ & B \rightarrow Aa/b \end{aligned}$$

$$\begin{aligned} \text{Ans} \quad & S_1 \rightarrow a/aA/B \\ & S \rightarrow a/aA/B \\ & A \rightarrow aBB \\ & B \rightarrow Aa/b/a \end{aligned}$$

$$\begin{aligned} \text{Now } X_1 &\rightarrow a \\ S_1 &\rightarrow a/X_1A/B \\ S &\rightarrow a/X_1A/B \\ A &\rightarrow X_1BB \\ B &\rightarrow Aa/b/a \end{aligned}$$

$$\begin{aligned} \text{Now } X_1 &\rightarrow a \quad X_2 \rightarrow BB \\ S &\rightarrow a/X_1A/b \quad \} B=b \\ S &\rightarrow a/X_1A/b \\ A &\rightarrow X_1X_2 \\ B &\rightarrow AX_1/b/a \end{aligned}$$

* # Chomsky Normal Form Conditions:

- (i) $A \rightarrow a$ where A, B and C are non terminals
(ii) $A \rightarrow BC$ & a is a terminal

Steps:

- 1) If start symbol S occurs on some right side, create a new start symbol S' & a new production $S' \rightarrow S$
- 2) Remove Null productions
- 3) Remove Unit productions
- 4) Replace each production $A \rightarrow B_1 \dots B_n$ where $n > 2$, with $A \rightarrow B_1C$ where $C \rightarrow B_2 \dots B_n$

Repeat this step for all productions having 2 or more symbols on the right side

- 5) If the right side of any production is in the form $A \rightarrow aB$ where $a = \text{terminal}$ & A and B are non terminals, then the production is replaced by $A \rightarrow XB$ and $X \rightarrow a$.

Repeat this step for every production which is of the form $A \rightarrow aB$.

Simplification of CFG -

Steps -

- 1) Reduction of CFG
- 2) Removal of Unit Productions
- 3) Removal of Null Productions

Reduction of CFG -

CFG are reduced in 2 phases.

Phase 1. Derivation of an equivalent grammar G' from the CFG G such that each variable derives some terminal string.

- 1) Include all symbols U_i that derives some terminal and initialize $i=1$
- 2) Include symbols U_{i+1} that derives U_i
- 3) Repeat increment i and repeat step 2, until $U_{i+1} = U_i$
- 4) Include all production rules that have U_i in it

Phase 2 Derivation of an equivalent grammar G'' from the CFG G' such that each symbol appears in a Derivation from

- 1) Include the start symbol in Y and initialize $i=1$
- 2) Include all symbols Y_{i+1} that can be derived from Y_i and include all production rules that have been applied.
- 3) Increment i and repeat step 2, until $Y_{i+1} = Y_i$

Q. Find a reduced grammar equivalent to the grammar G , having production rules

$P: S \rightarrow AC/B$
 $A \rightarrow a$
 $C \rightarrow c|BC$
 $E \rightarrow aA|e$

Ans Phase 1:

$T = \{a, c, e\}$
 $U_1 = \{A, C, E\}$

$U_2 = \{A, C, E, S\}$
 $U_3 = \{A, C, E, S\}$

$G' = \{A, C, E, S\}, \{a, c, e\}, P, \{S\}$

$P: S \rightarrow AC \text{ (B not included)}$

$A \rightarrow a$

$C \rightarrow c$

$E \rightarrow aA|e$

Phase 2:

$Y_1 = \{S\}$

$Y_2 = \{S, A, C\}$

$Y_3 = \{S, A, C, a, c\}$

$Y_4 = \{S, A, C, a, c\}$

$G'' = \{A, C, S\}, \{a, c\}, P, \{S\}$

$P: S \rightarrow AC$

$A \rightarrow a$

$C \rightarrow c$

Removal of Unit Productions -

$A \rightarrow B$ is called unit production

- 1) To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar $[x \in \text{Terminal}, x \text{ can be Null}]$
- 2) Delete $A \rightarrow B$ from the grammar
- 3) Repeat from step 1 until all unit productions are removed

Q. Remove unit productions from the Grammar

where production rule is given by

$P: S \rightarrow XY$

$X \rightarrow a$

$Y \rightarrow Z/b$

$Z \rightarrow M$

$M \rightarrow N$

$N \rightarrow a$

Ans

$Y \rightarrow ?$ } We need
 $X \rightarrow Z \rightarrow M$ } the remove
 $X \rightarrow M \rightarrow N$ } those

1) Since $N \rightarrow a$, we add $M \rightarrow a$ & remove $M \rightarrow N$

$P: S \rightarrow XY$

$X \rightarrow a$

$Y \rightarrow Z/b$

$Z \rightarrow M$

$M \rightarrow a$

$N \rightarrow a$

2) Since $M \rightarrow a$, we add $Z \rightarrow a$ & remove $Z \rightarrow M$

$P: S \rightarrow XY \mid Y \rightarrow Z/b \mid M \rightarrow a$

$X \rightarrow a$

$Z \rightarrow a$

$N \rightarrow a$

3) Since $Z \rightarrow a$, we add $Y \rightarrow a$ & remove $Y \rightarrow Z$

$P: S \rightarrow XY \mid Y \rightarrow a/b \mid M \rightarrow a$

$X \rightarrow a \mid Z \rightarrow a \mid N \rightarrow a$

4) But Z, M, N are unreachable from S

Remove them

$P: S \rightarrow XY \mid Y \rightarrow a/b$

$X \rightarrow a$

Removal of Null Productions

$A \rightarrow \epsilon = \text{Null Production}$

1) To remove $A \rightarrow \epsilon$, look for all productions where

right side contains A

2) Replace each occurrence of ' A ' in each of these

productions with ϵ

3) Add the resulting productions to the Grammar

Q. Remove Null Productions from the following

Grammar

~~Ans~~ $S \rightarrow ABAC$

$A \rightarrow aA/c$

$B \rightarrow bB/c$

$C \rightarrow c$

Ans $A \rightarrow \epsilon$ } We need to

$B \rightarrow \epsilon$ } remove this

1) To eliminate $A \rightarrow \epsilon$

$S \rightarrow ABAC$

2) Replace one A with ϵ on both A 's

$S \rightarrow AB\epsilon / BAC / BC$

3) Now for $A \rightarrow aA$, replacing A with ϵ

$A \rightarrow a$

4) New Production

$S \rightarrow ABAC / AB\epsilon / BAC / BC$

$A \rightarrow aA / a$

$B \rightarrow bB / \epsilon$

$C \rightarrow c$

5) To eliminate $B \rightarrow \epsilon$

$S \rightarrow ABAC / AC / BC$

$B \rightarrow b$

6) New Production

$S \rightarrow ABAC / AB\epsilon / BAC / BC / AC / AC / C$

$A \rightarrow aA / a, B \rightarrow bB / b, C \rightarrow c$

Conversion of CFG to CNF

Q. Convert following CFG to CNF:

P: $S \rightarrow ASA/aB$

$A \rightarrow B/S$

$B \rightarrow b/\epsilon$

Ans

1) Since S appears in RHS, we add a new state S' and $S' \rightarrow S$ is added to the production

P: $S' \rightarrow S$

$S \rightarrow ASA/aB$

$A \rightarrow B/S$

$B \rightarrow b/\epsilon$

2) Remove the Null Productions: $B \rightarrow \epsilon$ and $A \rightarrow \epsilon$

After removing $B \rightarrow \epsilon$:
P: $S' \rightarrow S$

$S \rightarrow ASA/aB/a$

$A \rightarrow B/S/\epsilon$

$B \rightarrow b$

3) Remove the Unit Productions: $S \rightarrow S, S' \rightarrow S, A \rightarrow B \& A \rightarrow S$

After removing $S \rightarrow S$:

P: $S' \rightarrow S$

$S \rightarrow ASA/aB/a/S/S/A$

After removing $S' \rightarrow S$:

P: $S' \rightarrow ASA/aB/a/S/S/A$

$S \rightarrow ASA/aB/a/S/S/A$

After removing $A \rightarrow B$:

P: $S' \rightarrow ASA/aB/a/S/S/A$

$S \rightarrow ASA/aB/a/S/S/A$

After removing $A \rightarrow S$:

P: $S' \rightarrow ASA/aB/a/S/S/A$

$S \rightarrow ASA/aB/a/S/S/A$

4) Now find out the productions that have more than 2 variables in RHS: $S' \rightarrow ASA, S \rightarrow ASA \& A \rightarrow ASA$

After removing these we get:

P: $S' \rightarrow AX/aB/a/S/S/A$

$S \rightarrow AX/aB/a/S/S/A$

$A \rightarrow b/AX/aB/a/S/S/A$

$B \rightarrow b$

$X \rightarrow SA$

5) Now change the productions $S' \rightarrow aB, S \rightarrow aB \& A \rightarrow aB$

P: $S' \rightarrow AX/YB/a/AS/SA$

$S \rightarrow AX/YB/a/AS/SA$

$A \rightarrow b/AX/YB/a/AS/SA$

$B \rightarrow b$

$X \rightarrow SA$

$Y \rightarrow a$

Greibach Normal Form

Conditions:

(i) $A \rightarrow b$

(ii) $A \rightarrow bC_1C_2 \dots C_n$

1) Check if the given CFG has any Unit or Null Productions and remove if there are any

2) Check whether the CFG is already in CNF

3) Convert it to CNF if it is not

Change the names of the Non Terminals

Symbol into some A_i in ascending order of i

Q. $S \rightarrow CA/BB \mid C \rightarrow b$

$B \rightarrow b/SB \mid A \rightarrow a$

Ans Replace: S with A_1

C with A_2

A with A_3

B with A_4

We get:

$$\begin{aligned} A_1 &\rightarrow A_1 A_3 | A_4 A_4 \\ A_4 &\rightarrow b | A_1 A_4 \\ A_2 &\rightarrow b \\ A_3 &\rightarrow a \end{aligned}$$

4) Alter the rules so that the non terminals are in ascending order, such that, if the production is of the form $A_i \rightarrow A_j x$, then $i < j$ & should never be $i \geq j$

$$A_4 \rightarrow b | \underline{A_1} A_4$$

$$A_4 \rightarrow b | A_4 A_4 A_4 | \underline{A_2} A_3 A_4$$

$$A_4 \rightarrow b | A_4 A_4 A_4 | b A_3 A_4$$

$\checkmark$ $\downarrow$ left $\rightarrow$ in GNF

Recursion

5) Remove left Recursion
Introduce a new variable to remove the left recursion

$$A_4 \rightarrow b | b A_3 A_4 | A_4 A_4 A_4$$

$$Z \rightarrow A_4 A_4 Z | A_4 A_4$$

$$\begin{aligned} \text{Add new variable} &\rightarrow A_4 \rightarrow b | b A_3 A_4 | b Z | b A_3 A_4 Z \\ \text{with out it} &\rightarrow \end{aligned}$$

Now grammar is:

$$A_1 \rightarrow A_2 A_3 | A_4 A_4$$

$$A_4 \rightarrow b | b A_3 A_4 | b Z | b A_3 A_4 Z$$

$$Z \rightarrow A_4 A_4 | \cancel{A_4 A_4} A_4 A_4 Z$$

$$A_2 \rightarrow b$$

$$A_3 \rightarrow a$$

$$\begin{aligned} A_1 &\rightarrow b A_2 | b A_4 | b A_3 A_4 A_4 | b Z A_4 | b A_3 A_4 Z A_4 \\ A_4 &\rightarrow b | b A_3 A_4 | b Z | b A_3 A_4 Z \\ Z &\rightarrow b A_4 | b A_3 A_4 A_4 | b Z A_4 | b A_3 A_4 Z b A_4 | \\ &\quad b A_4 Z | b A_3 A_4 A_4 A_4 Z | b Z A_4 Z | b A_3 A_4 Z A_4 Z \\ A_2 &\rightarrow b \\ A_3 &\rightarrow a \end{aligned}$$

CNF vs GNF

CNF	GNF
1) $A \rightarrow Bc$ $\left\{ \begin{array}{l} A, B, C \in \text{Variables} \\ a \in \text{Terminals} \end{array} \right.$	1) $A \rightarrow aX$ where $\{X \in V^* \mid a \in T\}$
OR $A \rightarrow a$	

2) No. of steps required to generate a string of length 'n' is " $2(n-1)$ "

3) Used in membership algorithm

4) Derivation can have tree obtained from CNF is always Binary tree

5) length of each production restricted

Pumping lemma for CFL -

Used to prove that a language is NOT context free
If A is a context free language, then A has a pumping length 'P' such that any string 'S', where $|S| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

- (i) $u^i x y^i z$ is in A for every $i \geq 0$
- (ii) $|v y| > 0$
- (iii) $|v x y| \leq P$

We prove using CONTRADICTION -

- 1) Assume that A is context free
- 2) It has to have a pumping length (say P)
- 3) All strings longer than P can be pumped $|S| \geq P$
- 4) Now find a string S in A such that $|S| \geq P$
- 5) Divide S into $uvxy^2z$
- 6) Show that $uv^ixy^iz \in A$ for some i
- 7) Then consider the ways that S can be divided into $uvxy^2z$
- 8) Show that none of these can satisfy all 3 conditions at the same time
- 9) S cannot be pumped $\Rightarrow$ CONTRADICTION

Q Show that $L = \{a^N b^N c^N \mid N \geq 0\}$ is not context free

Ans Assume that L is context free

L must have a pumping length

Now, we take a string S such that $S = a^P b^P c^P$

We divide S into parts $uvxy^2z$

eg. $P=4$

So, $S = a^4 b^4 c^4$

Case 1. v and y each contain only one type of symbol

$aaaa bbbb cccc$

$uv \quad x \quad y^2 \quad z$

uv^ixy^iz

$i=2$

uv^2xy^2z

$aaaaabbbbc cccc$

$a^6 b^4 c^5 \notin L$

Not context free

Case 2. Either v or y has more than one kind of symbol

$aaaa bbbb cccc$

$uv \quad vxy^2 \quad z$

uv^ixy^iz

uv^2xy^2z

$aaabbabb bbb cccc$

$a^6 b^7 c^4 \notin L$ (Not in $a^N b^N c^N$ form)

Q. Show that $L = \{wuw \mid w \in \{0,1\}^*\}$ is not context free.

Ans Assume L is context free

L must have a pumping length (say P)

Now, we take a string S such that $S = 0^P 1^P 0^P$

~~Case 1.~~ $P=5$

$S = 0^5 1^5 0^5$

Case 1. vxy does not straddle a boundary

$00000^x 11111^x 00000^x 11111$

$u \quad vxy \quad z$

$|vxy| \leq P$

uv^ixy^iz

uv^2xy^2z

0000011111000001111

$u \quad vxy^2 \quad z$

$0^5 1^7 0^5$

$\notin L$

Not equal

Case 2a. vxy straddles the first boundary

$00000^x 11111^y 00000^x 11111^z$

$u \quad vxy \quad z$

uv^2xy^2z

uv^2xy^2z

~~0000000000000000~~

00000001111110000011111

$0^7 1^9 0^5 1^5$

$\notin L$

Case 2b. vxy straddles the third boundary

$00000^x 11111^y 00000^x 11111^z$

$u \quad vxy \quad z$

uv^2xy^2z

uv^2xy^2z

00000111110000000011111

$0^5 1^5 0^7 1^9$

$\notin L$

Case 3. vxy straddles the midpoint

$00000^x 11111^y 00000^x 11111^z$

$u \quad vxy \quad z$

uv^2xy^2z

uv^2xy^2z

00000111111000000011111

$0^5 1^7 0^3 1^5$

$\notin L$

#

Pushdown Automata -

$P = (Q, \Sigma, \delta, q_0, z_0, F)$

δ takes as argument a triple $\delta(q, a, X)$ where -

(i) q is a state in Q

(ii) a is either an input symbol in Σ or $a = \epsilon$

(iii) X is a stack symbol, that is a member of Γ

The output of δ is finite set of pairs (p, Y) where:

p = new state

Y = string of stack symbols that replace X at the top of the stack

Eg: If $y = \epsilon$ then $y = \epsilon$ then stack is popped

If $y = X$ then stack is unchanged

If $y = YZ$ then X is replaced by Z and Y is pushed onto the stack

(A) $\xrightarrow{a} (B)$ Finite state machine

(A) $\xrightarrow{a^p b^c} (B)$ Pushdown Automata

Turnstile notation:

$\vdash$ sign decides

The turnstile notation

& represents 1 move

$\vdash^*$ sign decides a

sequence of moves

PDA can be accepted by -

(i) Acceptance by final state

(ii) Acceptance by empty stack

Q = input symbol $\rightarrow$ maybe ϵ

b = symbol on top of stack.

This symbol is popped

$\hookrightarrow \epsilon$ means the stack is

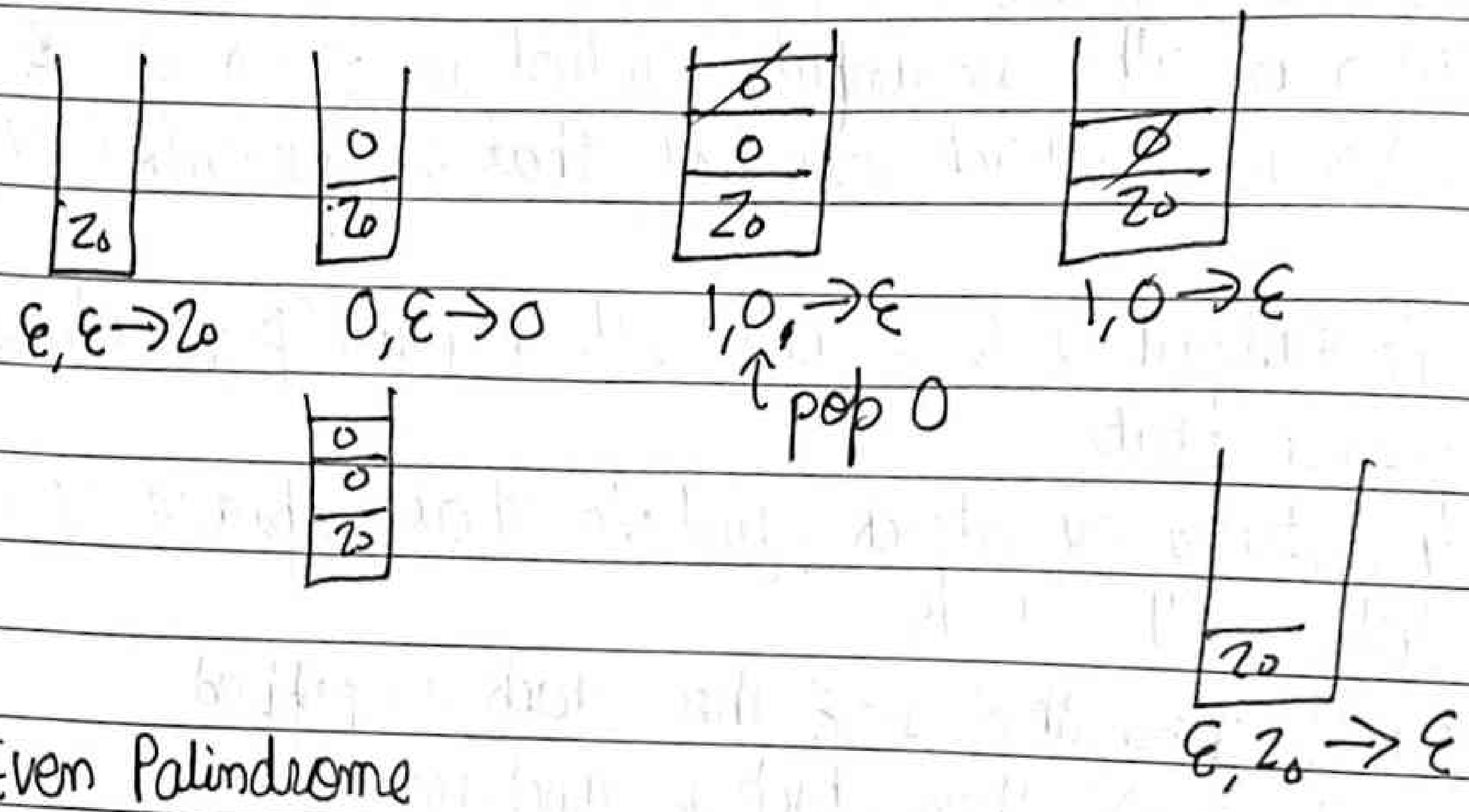
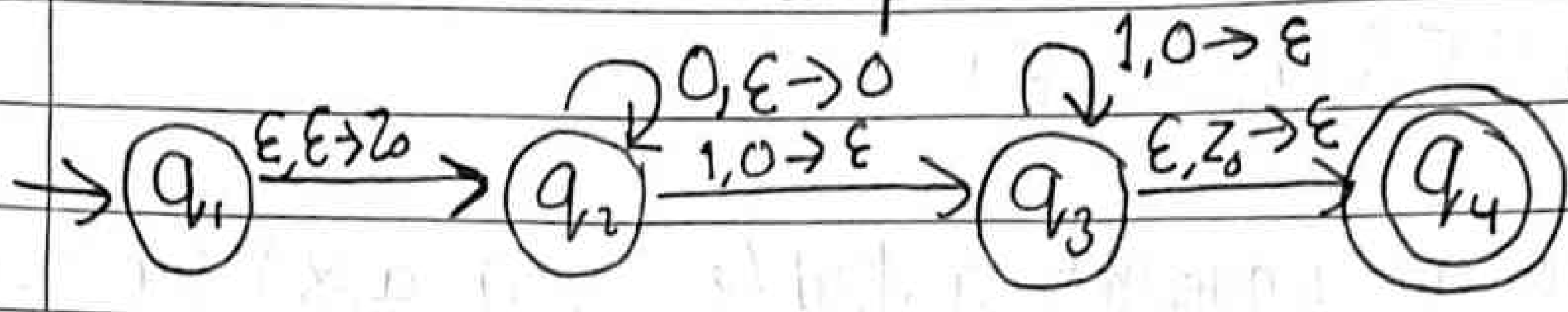
neither read or popped

c = this symbol is

pushed onto the stack

$\hookrightarrow \epsilon$ means nothing is pushed

Q. Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$



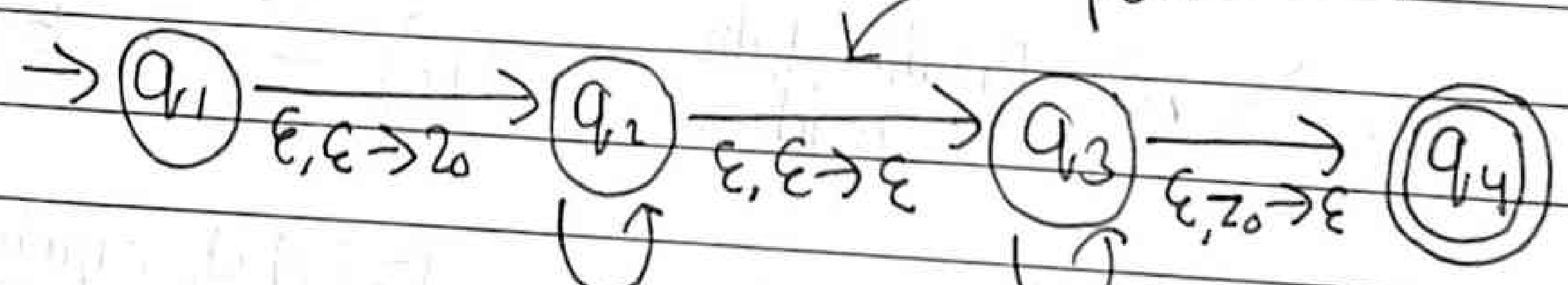
Q. Even Palindrome

Construct a PDA that accepts even palindromes of the form

$$L = \{ww^R \mid w = (a+b)^+\}$$

middle part NO | ON

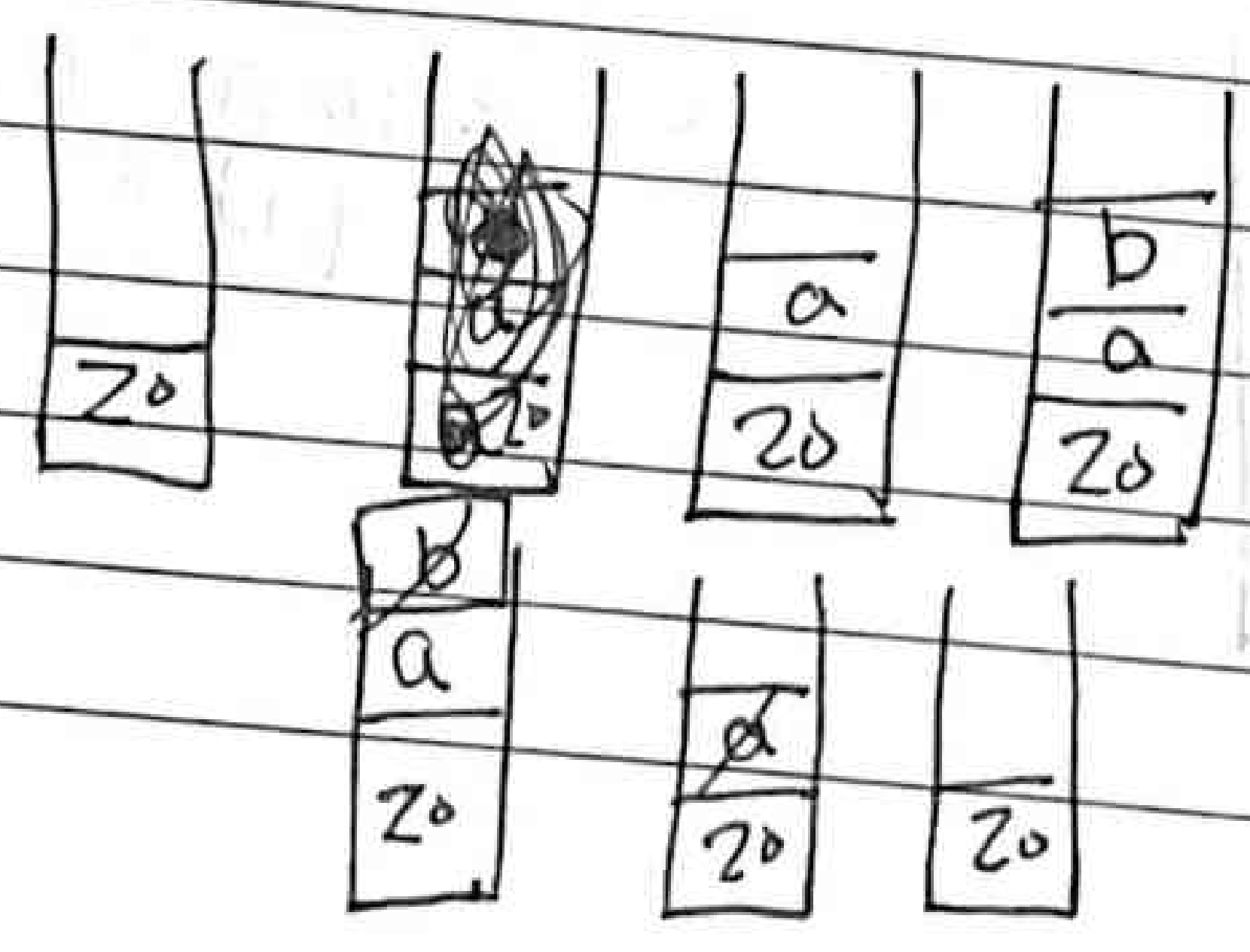
A.no



pushing the inputs a or b

pop a the top element if it matches with an input string

abba



Instantaneous Decryption (ID)-

Informal notation of how a PDA computes an input string and make a decision that string is accepted or rejected.

$ID(q, w, \alpha)$

q = decodes the current state

w = decodes the remaining input

α = decodes the stack contents, top at the left

Turnstile Notation-

$\vdash$ sign decodes the turnstile notation and represents one move

$\vdash^*$ decodes a sequence of moves

eg. $(p, b, T) \vdash (q, w, \alpha)$

In the above examples, while taking a transition from state p to q , the input symbol 'b' is consumed & top of the stack 'T' is represented by a new string α .

Pushdown Automata Acceptance-

There are 2 different ways to define PDA acceptability

- (i) Finite State Acceptability
- (ii) Empty Stack Acceptability

1) Finite State Acceptability-

PDA is said to accept its input by the final state if it enters any final state in zeros or more moves after reading the entire input

let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$ be PDA.
language accepted by final state can be defined as

$$L(PDA) = \{w \mid (q_0, w, z) \vdash^* (p, \epsilon, \epsilon), p \in F\}$$

2) Acceptance by Empty stack-

On reading the input string over, the initial symbol is same PDA, the stack of PDA gets empty.

$$N(PDA) = \{w \mid (q_0, w, z) \vdash^* (p, \epsilon, \epsilon), q \in Q\}$$

PDA has 2 types -

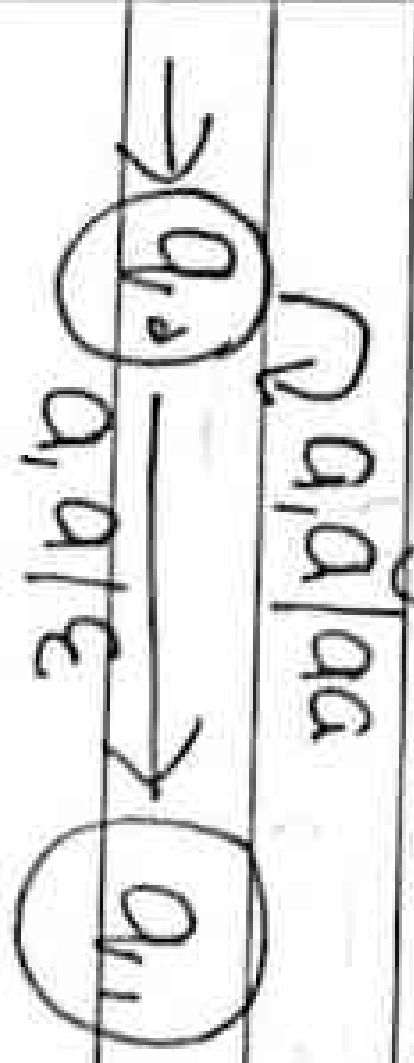
- (i) Deterministic PDA
- (ii) Non Deterministic PDA

Deterministic PDA

A PDA is deterministic if it satisfies the following conditions

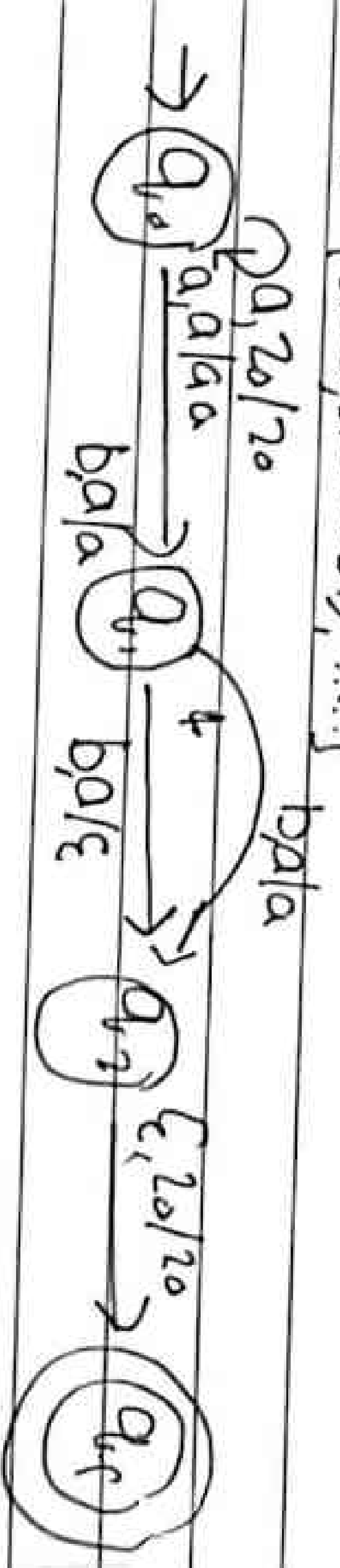
- (i) $\delta(q, a, x)$ has only one member for any given q in Q , a in Σ or $a = \epsilon$ and x in Σ .
- (ii) If $\delta(q, a, x)$ is non empty for some a in Σ then $\delta(q, \epsilon, x)$ must be empty

PDA is said to be deterministic if all derivations in the design has to give only given move



$$\text{eg- } L = a^n b^{2n} n > 0$$

$L = \{abbb, aabbbb, \dots\}$



For one transition, we can take more than one move

Diagram illustrating a genetic cross:

Parent 1 (Q₁) is crossed with Parent 2 (Q₁).

The resulting offspring are Q₁ and Q₂.

The cross is labeled with the genotypes Q₁ and Q₂ above the parents, and the resulting offspring Q₁ and Q₂ below the parents.

$Q_0 \xrightarrow{Q_1/a_{20}} Q_1$
 $Q_0 \xrightarrow{3/2, 1/2} Q_2$

$\begin{array}{c} \textcircled{q_0} \\ \downarrow a, b/a, b \\ q_1 \\ \downarrow a, a/e \\ q_2 \\ \downarrow b, b/b \\ q_3 \\ \downarrow e, a/z_0 \\ q_4 \end{array}$

- 1) Had at most one possible move for a given state, input symbol & stack symbol

3) Accepto input if it reaches

empty stack (if defined that way)

4) Not all CG's have an equivalent DPDA

5) Does not allow ~~that~~

6) $L = \{a^n b^n \mid n \geq 0\}$ recognised by DPDA

1) Can have multiple moves
for a given state,
input ~~and~~ symbol &
stack symbol.

2) Most powerful than

3) Accepts inputs if atleast one computation path reaches a final state on an empty stack.

4) Every CFG has an equivalent NPD?

5) Use non determinism to explore multiple paths

6) $L = \{a^n b^n c^n \mid n \geq 0\}$

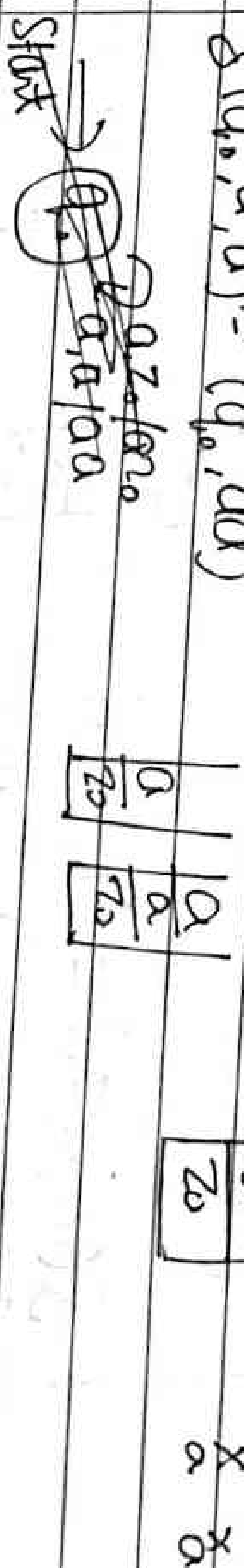
Recognised by NPDA but not PDA.

Q. 3

Given a PDA to accept the following language
 $L = \{a^n b^{2n} \mid n \geq 1\}$ by final state
 $\{abb, aabbbb, \dots\}$

$$\delta(q_0, a, z_0) = (q_0, a, z_0)$$

$$S(q, a, a) = (q, aa)$$



$\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa) \rightarrow q_0$ $\uparrow \uparrow \uparrow$ $aaabbb$
 $\delta(q_0, b, a) = (q_1, a)$ $\rightarrow$ No operation

$\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa)$
 $\delta(q_0, b, a) = (q_1, a)$

$\delta(q_1, b, a) = (q_2, \epsilon)$ $\rightarrow$ Pop
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_2, b, a) = (q_1, a)$
 $\delta(q_2, \epsilon, z_0) = (q_3, z_0)$
 $\delta(q_1, b, a) = (q_1, a)$
 $\delta(q_1, a, z_0) = (q_1, az_0)$

$\delta(q_1, b, a) = (q_1, a)$
 $\delta(q_1, \epsilon, z_0) = (q_3, z_0)$
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

Q. Construct a PDA for $L = \{a^n b^m c^n \mid n \geq 1, m \geq 1\}$

Ans. $\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa)$
 $\delta(q_0, b, a) = (q_1, a)$

$\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa)$
 $\delta(q_0, b, a) = (q_1, a)$

$\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa)$
 $\delta(q_0, b, a) = (q_1, a)$

$\delta(q_0, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_0, a, z_0) = (q_0, az_0)$
 $\delta(q_0, a, a) = (q_0, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

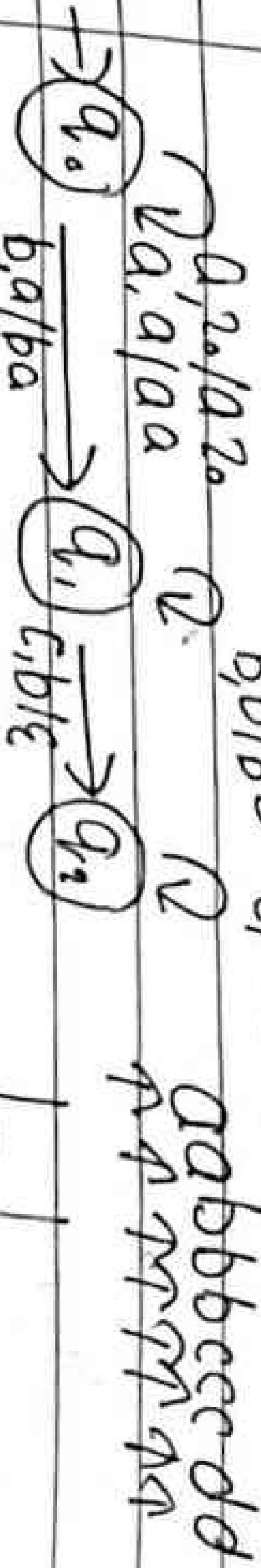
$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

$\delta(q_1, b, a) = (q_1, a)$ $\rightarrow$ No change
 $\delta(q_1, a, z_0) = (q_1, az_0)$
 $\delta(q_1, a, a) = (q_1, aa)$

Q. Construct PDA for $L = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$

Ans $n=2, m=3$

aa bbb ccc dd



$$\delta(q_0, a, z_0) = (q_0, a, a)$$

$$\delta(q_0, a, a) = (q_1, a, a)$$

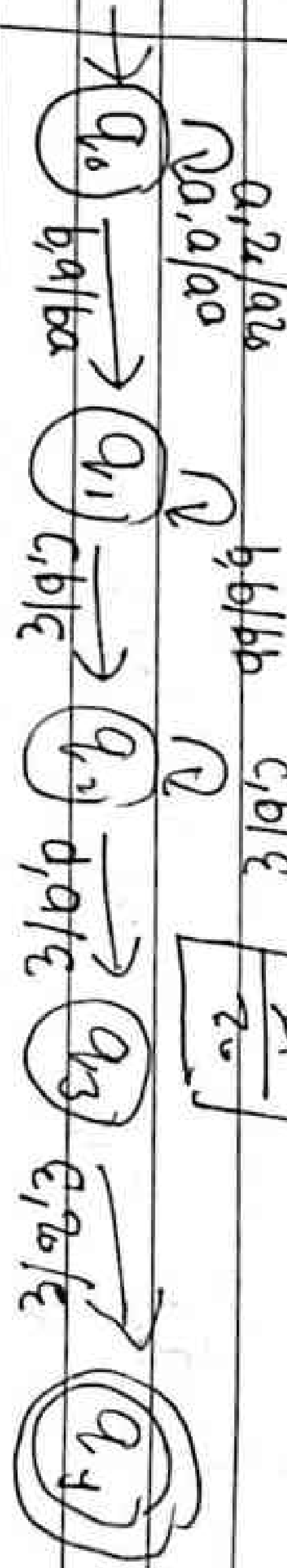
$$\delta(q_1, b, a) = (q_1, b, a)$$

$$\delta(q_1, b, b) = (q_1, b, b)$$

$$\delta(q_1, c, b) = (q_1, c)$$

$$\delta(q_2, c, b) = (q_2, c)$$

$$\delta(q_2, d, a) = (q_3, \epsilon)$$



$$\delta(q_3, d, a) = (q_3, \epsilon)$$

$$\delta(q_3, \epsilon, z_0) = (q_4, \epsilon)$$

Q. Construct a PDA for $L = \{0^n 1^n \mid n \geq 1\}$

Ans

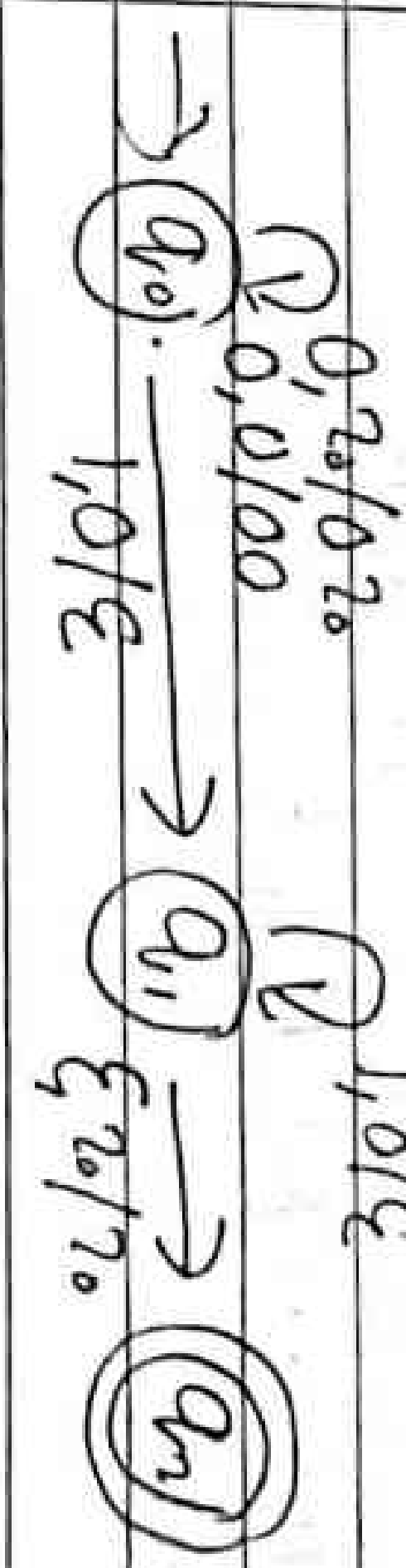
$L = \{01, 0011, 000111, \dots\}$

$$\delta(q_0, 0, z_0) = (q_0, 0, z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_1, 1, 0) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, z_0)$$



$$\delta(q_2, a, x) = (p, \alpha)$$

$$(q, a, w, x, B) \vdash (q, w, \alpha, B)$$

Let's use ID to verify if 0011 will be accepted or not

0011

write 0 was there in $\delta(q_0, 0, z_0)$

$$(q_0, 0011, z_0) \vdash (q_0, 011, 0, z_0)$$

$$\vdash (q_0, 11, 00, z_0) \quad 0 \text{ replaced by } 00$$

$$\vdash (q_1, 1, 0, z_0) \quad 0 \text{ popped}$$

$$\vdash (q_1, \epsilon, z_0) \quad 0 \text{ replaced by } \epsilon$$

$$\vdash (q_2, \epsilon, z_0)$$

Conversion of CFG to PDA -

let $G = (V, T, R, S)$ be a CFG, construct a PDA,

that accepts by empty stack

$PDA, P = (\{q\}, T, V \cup T, \delta, q, S)$

Transition Function δ

1) For each variable A ,
 $\delta(q, \epsilon, A) = \{ (q, \beta) \mid A \rightarrow \beta \text{ in } R \}$ - ①

2) For each terminal a ,
 $\delta(q, a, a) = (q, \epsilon)$ - ②

Q. Construct a PDA for the following CFG & test whether "abbabb" is in $N(P)$

$G = (\{S, A\}, \{a, b\}, R, S)$

$R = \{ S \rightarrow AB, A \rightarrow SA \mid b \}$

Ans PDA, $P = (\{q\}, \{a, b\}, \{S, A, a, b\}, \delta, q, S)$

$\delta(q, \epsilon, S) = \{ (q, AA), (q, a) \}$ (from ①)

$\delta(q, \epsilon, A) = \{ (q, SA), (q, b) \}$ (from ①)

$\delta(q, a, a) = (q, \epsilon)$

$\delta(q, b, b) = (q, \epsilon)$

1D
 $(q, abbabb, S) \vdash (q, abbabb, AA)$

$\vdash (q, abbabb, SA)$

$\vdash (q, abbabb, a)$

$\vdash (q, abbabb, AA)$

$\vdash (q, abbabb, AA)$

$\vdash (q, abbabb, bA)$

$\vdash (q, abbabb, bA)$

1D $\vdash (q, abbabb, S) \vdash (q, abbabb, AA)$

Check if $\delta(q, a, S)$ is there

If not, check for $\delta(q, \epsilon, S)$

Now check for $\delta(q, a, A)$

else $\delta(q, \epsilon, A)$

$\vdash (q, abbabb, SA)$

Now, consider $\delta(q, \epsilon, S)$

$\vdash (q, abbabb, AA)$

Now, consider $\delta(q, a, A)$

$\vdash (q, abbabb, bA)$

Now, consider

$\delta(q, \epsilon, A)$

$\vdash (q, abbabb, bA)$

Now, consider

$\delta(q, b, b)$

$\vdash (q, abbabb, A)$

Similarly do this

continuously till

you reach

$\vdash (q, \epsilon, \epsilon)$

Q. Construct non deterministic PDA that for

$$L = \{wuw^R \mid w \text{ is in } (0+1)^+\}$$

Ans

$$\delta(q_0, 0, z_0) = (q_0, 0z_0) \quad \uparrow \quad \text{XXX}$$

$$\delta(q_0, 1, z_0) = (q_0, 1z_0) \quad \uparrow \quad \text{XXX}$$

$$\delta(q_0, 1, 0) = (q_0, 10)$$

$$\delta(q_0, 0, 1) = (q_0, 01)$$

$$\delta(q_0, 1, 1) = (q_0, 11)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, \epsilon, 1) = (q_1, 10)$$

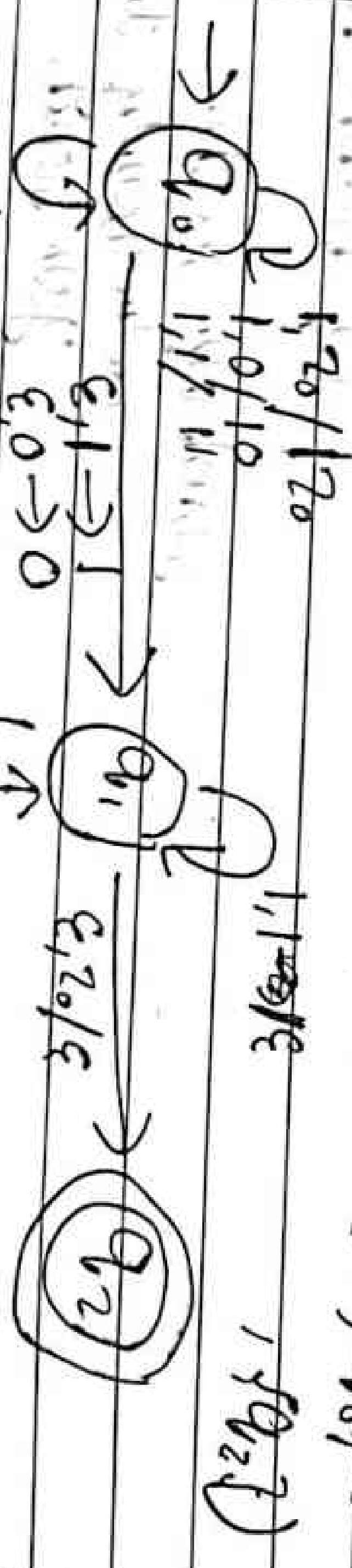
$$\delta(q_0, \epsilon, 0) = (q_1, 0)$$

$$\delta(q_1, 1, 1) = (q_1, \epsilon)$$

$$\delta(q_1, 0, 0) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$$

$$PDA, P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, z_0\}, \delta, q_0, z_0)$$



$$0, 0/0z_0$$

$$0, 1/01$$

$$0, 0/00$$

Expression Power $FA < PDA < LBA < TM$

Context Sensitive Language (CSL)

- When all the productions are of form

$$\alpha \rightarrow \beta \quad \text{where } \alpha \rightarrow (V+T)^* \quad V (V+T)^*$$

$$|\alpha| \leq |\beta| \quad \beta \rightarrow (V+T)^*$$

where α & β are strings of non terminals & terminals

- More powerful than CFG but less power than unrestricted grammar

- It has 4 tuples - where, $N/V =$ set of non terminals

$$G = (N, \Sigma, P, S)$$

on $\Sigma/T =$ set of terminals $P =$ finite set of productions

Properties: $S =$ start symbol

(i) Union, Intersection of 2 context sensitive language is context sensitive

(ii) Complement of CSL is also CSL

Q. Consider the following CSG -

$$S \rightarrow abc / a^k b^k$$

$$Ab \rightarrow bA$$

$$Ac \rightarrow Bb c$$

$$Bb \rightarrow Bb$$

$$aB \rightarrow aa / aA$$

What is the language generated by the grammar

Ans

$$S \rightarrow a^k b^k c^k \quad \text{or} \quad a^k a b^k b c^k$$

$$a b A c \quad a a b b A c c$$

$$a b B b c c \quad a a b b B b c c c$$

$$a b B b c c \quad a a b b B b c c c$$

$$a a A b b c c a \quad a a b b b b c c c c$$

Language generated by this grammar is $\{a^n b^n c^n \mid n \geq 1\}$

Linear Bound Automata (LBA) -

- Non deterministic TM which has a single tape whose length is not infinite but bounded by a linear function of the length of input string

- Recognizes CSG

- Grammar

$$G = \{Q, \Sigma, \delta, q_0, q_1, q_R, F\}$$

where, Q = set of all states

Σ = set of all terminals

E = input alphabet

δ = transitions

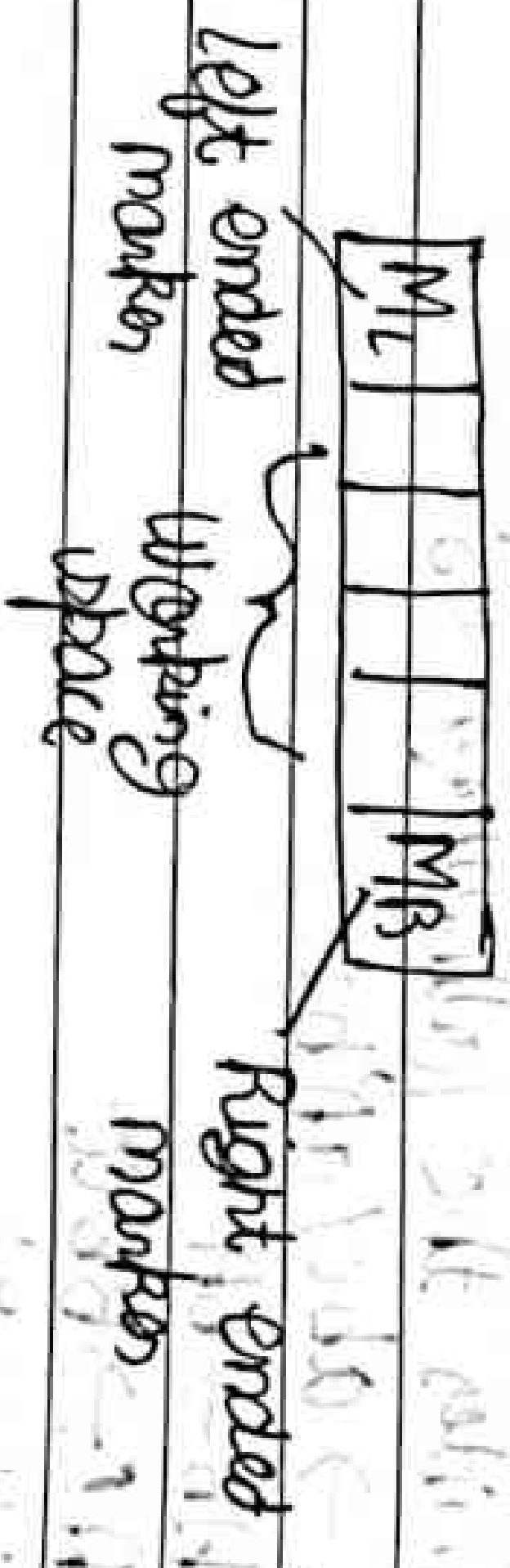
q_0 = initial state

q_1 = left bound tape

q_R = right bound tape

F = set of final state

$$LBA = TM + \text{Input size, tape}$$



- Behaves as a Turing Machine but storage space of tape is restricted only to length of inputs.
- Less powerful than TM but more powerful than PDA

- Computation restricted to constant bounded area
- Input alphabet contains 2 special symbols which serve as left end marker and right end

marker, which means the transition neither moves left of the left end marker nor right of the right end marker

- Applications -

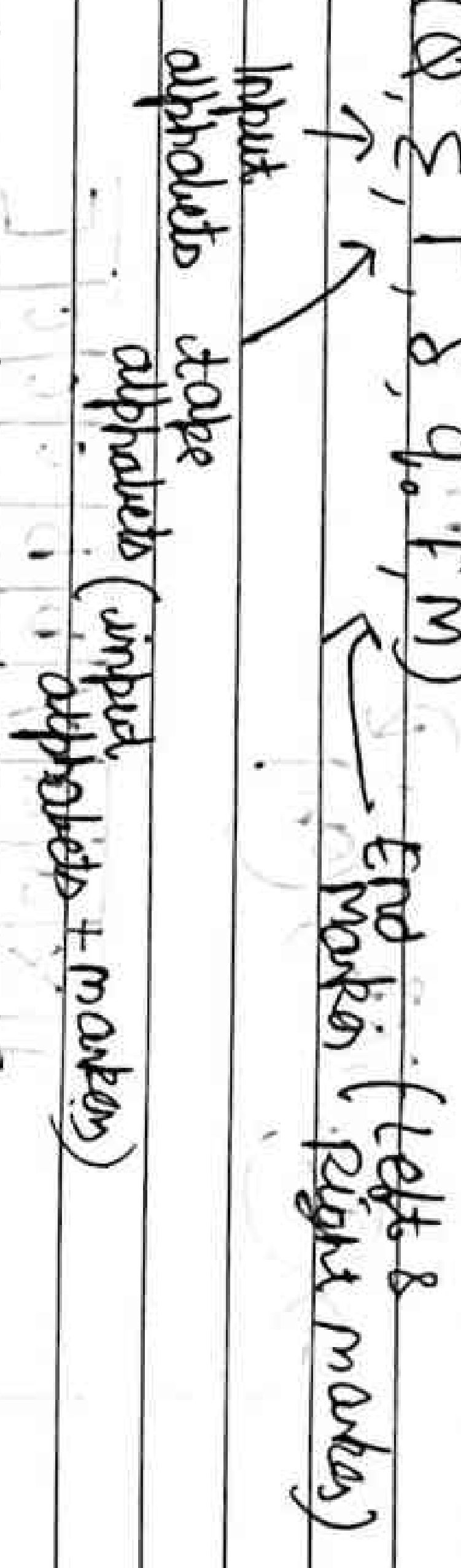
- For implementation of genetic programming
- For constructing syntactic parse tree for semantic analysis for compiler

* Relation between LBA & CSL -

- Set of strings accepted by non deterministic LBA is the set of strings generated by the CSL, excluding null strings
- If L is a context sensitive language, then L is accepted by a LBA

7 tuple Grammar

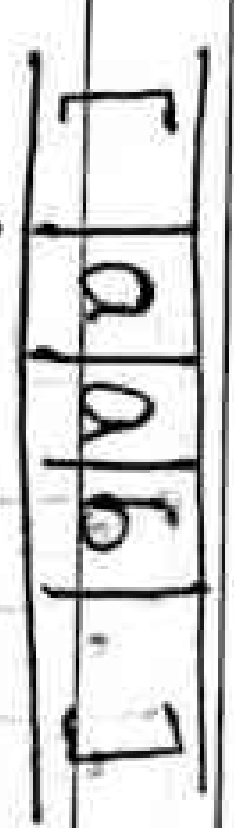
$$(Q, \Sigma, \Gamma, \delta, q_0, F, M)$$



$$\delta: Q \times \Gamma \rightarrow Q \times \{L, R\}$$



Model:



Q Design LRN for $a^n b^n c^n$ in shift reduction

Ans
a a a b b b c c c
x x x y y y z z z

$\boxed{X|a|a|a|b|b|b|c|c|c|}$

$\rightarrow q_0 \xrightarrow{a, x, R} q_1$

$\boxed{X|X|a|a|b|b|b|c|c|c|}$

$\rightarrow q_0 \xrightarrow{a, x, R} q_1 \xrightarrow{a, a, R} q_2$

$\boxed{X|X|a|a|y|b|b|b|c|c|c|}$

$\rightarrow q_0 \xrightarrow{a, x, R} q_1 \xrightarrow{b, y, R} q_2$

o, a, R

$\boxed{X|X|a|a|y|b|b|c|c|c|}$

$\rightarrow q_0 \xrightarrow{a, x, R} q_1 \xrightarrow{b, y, R} q_2 \xrightarrow{o, a, R} q_3$

$\boxed{X|X|a|a|y|b|b|z|c|c|c|}$

$\rightarrow q_0 \xrightarrow{a, x, R} q_1 \xrightarrow{b, y, R} q_2 \xrightarrow{c, z, L} q_3$

y, y, R

$q_4 \xrightarrow{z, z, R} q_5$

$q_5 \xrightarrow{z, z, R} q_6$

$\boxed{X|X|X|a|y|b|b|z|z|z|}$

$\boxed{X|X|X|X|y|y|y|z|z|z|}$